

Compiladores

Análise Léxica (*Scanning*)

Ciência da Computação / 6ª etapa

Prof. Dr. Leandro Carlos Fernandes





Objetivos

Nesta aula temos como meta:

- Iniciar o processo de análise de código;
- Entender as tarefas e compromissos da análise léxica;
- Compreender a estrutura dos lexemas, sua representação com elementos regulares e o uso de autômatos na tarefa de reconhecimento dos elementos significativos da linguagem;
- Aprender como implementar um analisador para identificar números, identificadores e operadores.



1ª fase do processo de Análise

ANÁLISE LÉXICA (SCANNING)

Análise Léxica

O QUE É? Lida com as técnicas para especificar e implementar *analísadores léxicos*.

TAREFA: esquadrinhar o código fonte, coletando os caracteres e agrupando-os logicamente de acordo com um **padrão**; e classificá-los, conferindo um **código interno** de acordo com sua estrutura.

- Os **agrupamentos lógicos** podem ser gerados por sequencia de caracteres (**lexemas**) diferentes.
- Os **códigos internos** desses agrupamentos são chamados de **tokens** (ou **átomos**).



Token ou átomo

Na maioria das linguagens de programação, são considerados átomos os seguintes agrupamentos lógicos:

Palavras-chaves, operadores, identificadores, constantes, strings, símbolos de pontuação (vírgula, ponto, ponto e vírgula, etc)

Note que **diferentes lexemas** (sequência de caracteres) podem gerar um mesmo **token**. Ou seja, um conjunto de diferentes caracteres podem determinar o mesmo átomo.



Funções do Analisador Léxico



Extração e classificação de átomos



Eliminação de delimitadores e comentários



Conversão numérica



Identificação de palavras reservadas



Tratamento de identificadores



Recuperação de erros



Interação com o sistema de arquivos



Controle da numeração de linhas do programa fonte

Tarefas



Esquadrinhar o código fonte, compondo tokens e classificando-os;



Reconhecer e validar os elementos identificadores;

Eliminar elementos desnecessários ao processo de compilação;

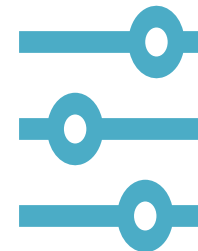


Prover um mecanismo para controle de erros amigável;



10

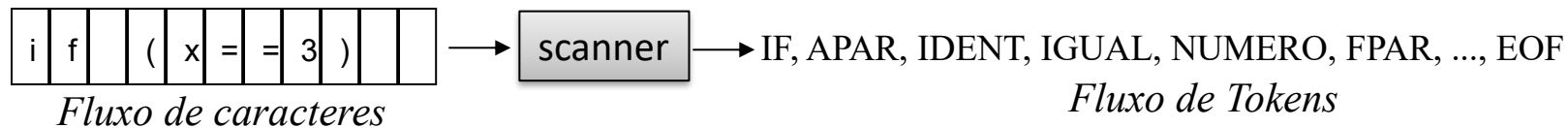
Reconhecer e consistir números inteiros e reais;



Tratar parâmetros para compilação condicional;

Tarefas de um Analisador Léxico

- Produzir símbolos terminais



- Ignorar e descartar símbolos irrelevantes
 - espaços em branco
 - caracteres de tabulação
 - caracteres de controle (CR e LF)
 - comentários

Por quê o analisador léxico não é uma parte do analisador sintático?

- Tokens possuem uma estrutura sintática

identif = letra {letra | dígito}

número = dígito {dígito}

if = "i""f"

igual = "="

...

Por quê o analisador léxico não é uma parte do analisador sintático?

- Isso deixaria o analisador sintático mais complicado de ser construído
 - Ex. dificulta a distinção entre palavras reservadas e identificadores

```
Statement = ident "=" Expr ";"  
           | "if" "(" Expr ")" ... .
```

- Precisaria ser reescrito para a forma:

```
Statement = "i" ( "f" "(" Expr ")" ...  
                | notF {letter | digit} "=" Expr ";"  
                )  
           | notI {letter | digit} "=" Expr ";" .
```

- O *scanning* deve eliminar brancos, tabulações, fins de linha e comentários

- Esses caracteres podem ocorrer em qualquer lugar do código, levando a gramáticas muito complexas

```
Statement = "if" {Blank} "(" {Blank} Expr {Blank} ")" {Blank} ... .  
Blank = " " | "\r" | "\n" | "\t" | Comment.
```

- Tokens podem ser descritos por linguagens regulares
 - mais simples e mais eficientes que as gramáticas livres de contexto

Gramáticas Regulares

Definição:

- Um gramática é dita regular se puder ser descrita por produções na forma:

$A = a$	$a, b \in \text{Símbolos Terminais}$
$A = b B$	$A, B \in \text{Símbolos Não-terminais}$

- Ex: Gramática de nomes

Identif = letra
letra Restante
Restante = letra
dígito
letra Restante
dígito Restante

Ex: Derivação do nome xy3

Identif
 ?
 letra Restante
 ?
 letra letra Restante
 ?
 letra letra dígito

Definição alternativa

- Um gramática é chamada regular se puder ser descrita por meio de uma simples e não recursiva produção EBNF.

$\text{Ident} = \text{letra} \{ \text{letra} \mid \text{dígito} \}$

Limitações das Gramáticas Regulares

- Gramáticas regulares não podem lidar com estruturas aninhadas
 - Não são capazes de manipular recursão central!
- Porém esse tipo de construção é importante na maioria das linguagens de programação

Expressões aninhadas

`Expr ?... "(" Expr ")" ...`

Comandos aninhados

`Comando ? "do" Comando "while" "(" Expr ")"`

Classes aninhadas

`Classe ? "class" "{" ... Classe ... "}"`

- Para construções desse tipo precisamos de Gramáticas Livres de Contexto

- A maioria das estruturas léxicas são regulares

Nomes

`letra { letra | dígito }`

Números

`dígito { letra | dígito }`

Strings

`"\"" { qqCaractereExcetoAspas } "\""`

Palavras reservadas

`letra { letra }`

Operadores

`"_~" | "_" | "." | "," | ...`

Exceto comentários aninhados:

`/* ... /* ... */ ... */`

o scanner deve manipulá-los de uma maneira especial

Expressões Regulares

- Notação alternativa para Gramáticas Regulares

Definição

- ε (cadeia vazia) é uma expressão regular
- um símbolo terminal é uma expressão regular
- Se α e β são expressões regulares as expressões abaixo também são regulares:

$\alpha\beta$

$(\alpha|\beta)$

$(\alpha)?$

$\varepsilon | \alpha$

$(\alpha)^*$

$\varepsilon | \alpha | \alpha\alpha | \alpha\alpha\alpha | \alpha\alpha\alpha\alpha | \dots$

$(\alpha)^+$

$\alpha | \alpha\alpha | \alpha\alpha\alpha | \alpha\alpha\alpha\alpha | \dots$

Exemplos:

- "w" "h" "i" "l" "e" while
- letra { letra | dígito }^{*} nomes
- dígito⁺ números

Autômatos Finitos Determinísticos

- Podem ser usados para analisar linguagens regulares

- Exemplo:

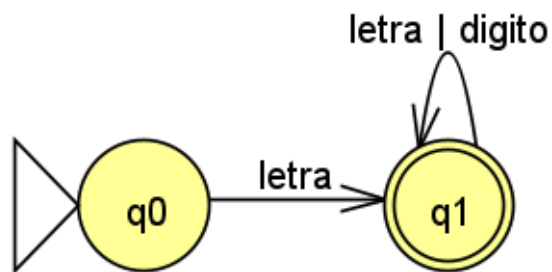


Tabela da **função de transição de estados**:

d	letra	dígito	"finito", porque d possui uma quantidade finita de estados possíveis
q0	q1	erro	
q1	q1	q1	

- Um autômato finito determinístico é um quintupla (S, I, δ, s_0, F)

S conjunto de estados

I alfabeto de entrada

$\delta: S \times I \rightarrow S$ função de transição

s_0 estado inicial

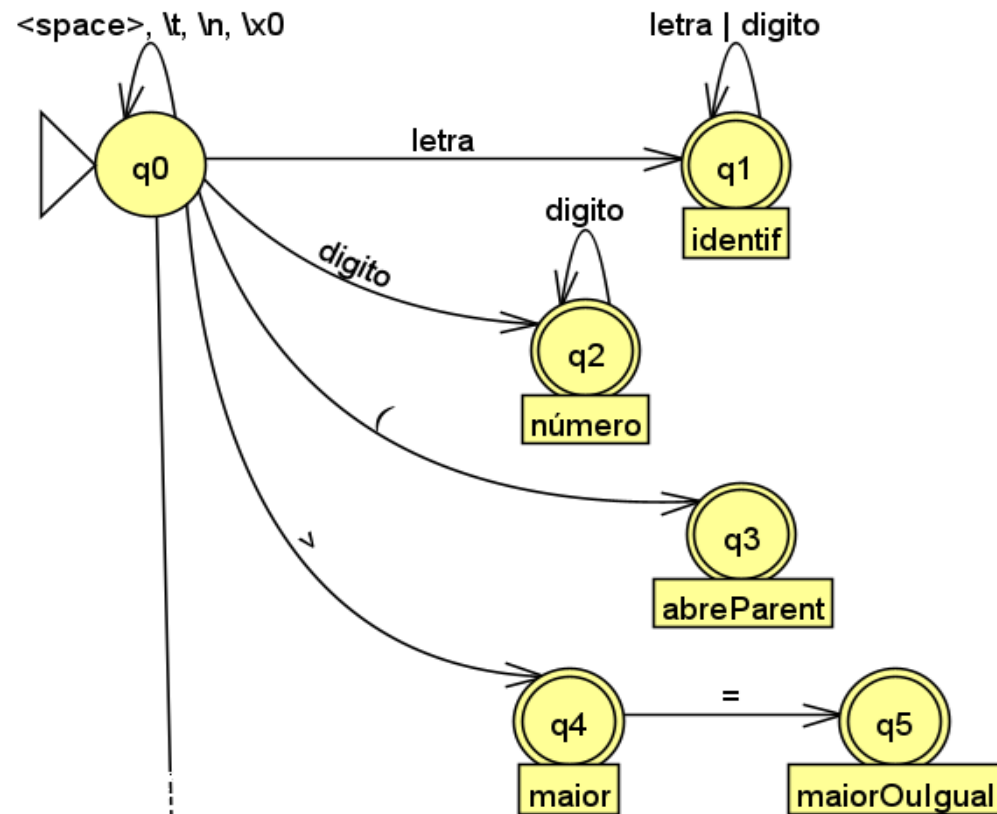
F conjunto de estados finais

- Um AFD terá reconhecido a sentença se:
 - estiver em um estado final
 - e se a entrada tiver sido consumida totalmente ou não for possível realizar uma transição com o próximo símbolo da entrada

A **linguagem** reconhecida por um AFD é o conjunto de todas as seqüências de símbolos que levam o autômato do estado inicial até um dos estados finais

O scanner como um AFD

- O analisador léxico pode ser visto como um grande AFD



Exemplo p/ entrada: `max >= 30`

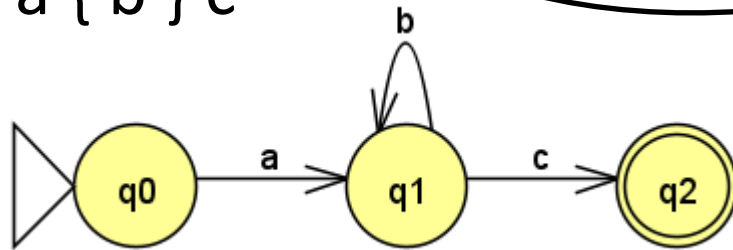
- $q0 \rightarrow q1$ segue reconhecendo as letras até que, sem transição com " " em q1, é reconhecido como **Ident**
- $q0 \rightarrow q5$ ignorando os brancos do início, segue até q4 e q5. Sem transições possíveis para o " ", para em q5 e é reconhecido como **maiorIgual**
- $q0 \rightarrow q2$ ignora os brancos do início e vai compondo dígito-a-dígito até que, sem transição com " " em q2, o lexema é reconhecido **número**

- Depois de reconhecer um token, o scanner parte do estado q0 novamente.

Implementando um AFD (ver 1)

Exemplo:

$A = a \{ b \} c$



Este é um exemplo de algoritmo universal de reconhecimento

- Implementação de δ como uma matriz

δ	a	b	c	
0	1	-	-	
1	-	1	2	
2	-	-	-	F

`int[,] delta = { {1, -1, -1},
 {-1, 1, 2},
 {-1, -1, -1} };`

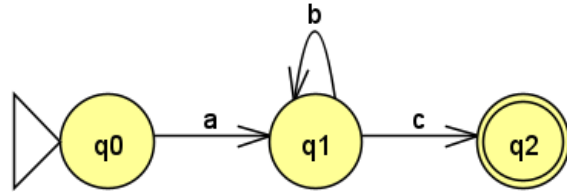
```
int[,] delta = new int[maxStates,  
                        maxSymbols];  
  
int lastState, state = 0; // DFA starts  
in state 0  
do {  
    int sym = next symbol;  
    lastState = state;  
    state = delta[state, sym];  
} while (state != undefined);  
assert(lastState, F); // is a final state?  
return recognizedToken[lastState];
```

Essa implementação pode ser muito ineficiente para um analisador real.

Implementando um AFD (ver 2)

Exemplo:

$A = a \{ b \} c$



- Codificação direta dos estados:

```
char ch = read();
s0: if (ch == 'a') { ch = read(); goto s1; }
    else goto err;
s1: if (ch == 'b') { ch = read(); goto s1; }
    else if (ch == 'c') { ch = read(); goto
s2; }
    else goto err;
s2: return A;
err: return errorToken
```

- Que em Java poderia ser algo mais ou menos assim:

```
int state = 0;
loop:
    for (;;) {
        char ch = read();
        switch (state) {
            case 0: if (ch == 'a') { state = 1; break; }
                    else break loop;
            case 1: if (ch == 'b') { state = 1; break; }
                    else if (ch == 'c') { state = 2;
break; }
                    else break loop;
            case 2: return A;
                    }
        }
    }
return errorToken;
```


Leitura sugerida



Leia o capítulo 2 do livro do Louden:

LOUDEN, Kenneth C. **Compiladores: princípios e práticas**.
Cengage Learning Brasil, 2004.

Leia os capítulos 4 e 5 do livro:

BARBOSA, Cynthia da S.; LENZ, Maikon L.; LACERDA, Paulo
S. Pádua D.; et al. **Compiladores**. Grupo A, 2021.





Universidade Presbiteriana
Mackenzie



Faculdade de
Computação e Informática

